Web Science

Assignment – 6

Institute of Web Science and Technologies

Universität Koblenz-Landau

Group name: FOWLER

Name: Jeevan Saraf

jeevansaraf@uni-koblenz.de

Matriculation no: **222100445**

Name: Parth Chavda

parthchavda@uni-koblenz.de

Matriculation no: **222100495**

Name: Prajna Shetty

prajnashetty73@uni-koblenz.de

Matriculation no: **222100479**

Name: Ritika Bhandari

rbhandari@uni-koblenz.de

Matriculation no: **222100563**

1 Zipf

You are provided with a text file (Data.txt).

- Calculate the rank of the words in the corpus (3.5 points)
- Calculate the frequency of the words in the corpus (3.5 points)
- Plot them to show if they hold Zipf's law. Validate your graph with a suitable explanation. (3 points)

Answer:

```
from operator import itemgetter

import matplotlib.pyplot as plt



freq = {}



with open("data.txt") as file:

    for i in file:

        words = i.lower().split()

        for word in words:

            freq[word] = freq.get(word,0) + 1



rank =sorted(freq.items(), key=itemgetter(1), reverse=True)



print("\n\nRank of the words in the Corpus: ", len(rank))

print("\nTop 10 most frequent Words in the corpus
:\n",rank[0:10])
```

values=[x[1] for x in rank]

plt.loglog(values)

plt.xlabel('Rank')

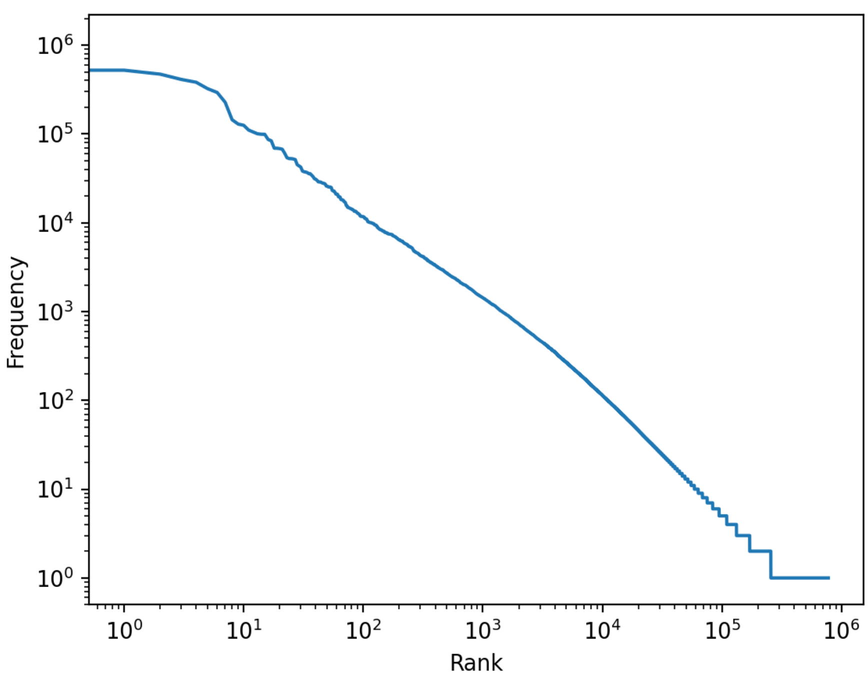
plt.ylabel('Frequency')

plt.show()

```
(base) jeevan@Jeevans-MacBook-Air Downloads % python task1ass6.py


Rank of the words in the Corpus:  775431

Top 10 most frequent Words in the corpus :
 [('the', 1095284), ('of', 523310), ('in', 472395), ('and', 412173), ('a', 382958), ('is', 323771),
 ('to', 293747), ('was', 227599), ('it', 144734), ('he', 129353)]
2022-12-14 22:06:40.107 python[96707:2015136] +[CATransaction synchronize] called within transactio
n
(base) jeevan@Jeevans-MacBook-Air Downloads %
```



From above graph we can show that our dataset holds the Zipf's Law which is basically a statistical distribution in datasets, such as words in a corpus, in which the frequencies of words are inversely proportional to their ranks.

2 Similarity Measurement I

You are provided with three documents(Di)below.

D1 = this is a text about web science

D2 = web science is covering the analysis of text corpa

D3 = scientific methods are used to analyze web pages

Write a short python script that calculates the Jaccard similarity between all the

Documents

Answer:

```
def similarityCalculator(a, b):
    return float(len(set(a).intersection(b))) / float(len(set(a).union(b))

D1 =  "this is a text about web science".split()

D2 =  "web science is covering the analysis of text corpa".split()

D3 =  "scientific methods are used to analyze web pages".split()


print("Jaccard similarity:")

print("Between D1 and D2:" , similarityCalculator(D1,D2))

print("Between D2 and D3:" , similarityCalculator(D2,D3))

print("Between D3 and D1:" , similarityCalculator(D3,D1))
```

```
Jaccard similarity:
Between D1 and D2: 0.3333333333333333
Between D2 and D3: 0.0625
Between D3 and D1: 0.07142857142857142
```

3 Vector Modeling (5 points)

Model the following three documents as vectors in the word vector space.

D1 = i love you

D2 = you are beautiful

D3 = i love beautiful koblenz

Answer the following questions:

1. How many base vectors would be needed to model the documents of this corpus? (0.5 point)

Answer: The words in the given corpus are considered as the base vectors. So, the corpus has 6 base vectors.

Base vectors => (i
love
you
are
beautiful
koblenz)

2. What does each dimension of the vector space stand? (0.5 point)

Answer:

$$\vec{i} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad \vec{love} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad \vec{you} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\vec{are} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad \overrightarrow{beautiful} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \quad \overrightarrow{Koblenz} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

3. Represent the documents as document vectors in the word vector space. Write complete formulas and equations with valid vector notations. (4 points)

$D_1 =$ i love you $\quad D_2 =$ You are beautiful $\quad D_3 =$ I love beautiful Koblenz

- $tf(i, D_1) = 1$
- $tf(love, D_1) = 1$
- $tf(you, D_1) = 1$

- $tf(you, D_2) = 1$
- $tf(are, D_2) = 1$
- $tf(beautiful, D_2)\ 1$

- $tf(1, D_3) = 1$
- $tf(love, D_3) = 1$
- $tf(beautiful, D_3) = 1$
- $tf(Koblenz, D_3) = 1$

Document vectors:

$$\vec{d_1} = \sum_{i=1}^{3} tf(w_i, D_1)\vec{w_i} = tf(i, D_1)\vec{i} + tf(love, D_1)\vec{love} + tf(you, D_1)\vec{you}$$

$$= 1\begin{pmatrix}1\\0\\0\\0\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\1\\0\\0\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\0\\1\\0\\0\\0\end{pmatrix} \Rightarrow \begin{pmatrix}1\\1\\1\\0\\0\\0\end{pmatrix}$$

$$\vec{d_2} = \sum_{i=1}^{3} tf(you, D_2)\vec{you} + tf(are, D_2)\vec{are} + tf(beautiful, D_2)\vec{beautiful}$$

$$= 1\begin{pmatrix}0\\0\\1\\0\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\0\\0\\1\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\0\\0\\0\\1\\0\end{pmatrix} \Rightarrow \begin{pmatrix}0\\0\\1\\1\\1\\0\end{pmatrix}$$

$$\vec{d_3} = tf(i, D_3)\vec{i} + tf(love, D_3)\vec{love} + tf(beautiful, D_3)\vec{beautiful} + tf(Koblenz, D_3)\vec{Koblenz}$$

$$= 1\begin{pmatrix}1\\0\\0\\0\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\1\\0\\0\\0\\0\end{pmatrix} + 1\begin{pmatrix}0\\0\\0\\0\\1\\0\end{pmatrix} + 1\begin{pmatrix}0\\0\\0\\0\\0\\1\end{pmatrix} \Rightarrow \begin{pmatrix}1\\1\\0\\0\\1\\1\end{pmatrix}$$

4 Similarity Measurement II (10 points)

Write a simple python script that computes the Cosine Similarity of the three documents defined in the previous question.

Answer:

```
import numpy as np
```

```
D1 = "i love you"

D2 = "you are beautiful"

D3 = "i love beautiful koblenz"


print("3 Documents\nD1 = {}\nD2 = {}\nD3 = {}\n---
".format(D1,D2,D3))


words1 = D1.lower().strip().split()

words2 = D2.lower().strip().split()

words3 = D3.lower().strip().split()


uniques = set(words1 + words2 + words3)


x = [0] * len(uniques)

y = [0] * len(uniques)

z = [0] * len(uniques)


for j, word in enumerate(uniques):

    x[j] = words1.count(word)

    y[j] = words2.count(word)

    z[j] = words3.count(word)
```

```python
x = np.array(x)

y = np.array(y)

z = np.array(z)



cos_d1_d2 = np.dot(x, y) / (np.sqrt(np.dot(x, x)) *
np.sqrt(np.dot(y, y)))

cos_d2_d3 = np.dot(y, z) / (np.sqrt(np.dot(y, y)) *
np.sqrt(np.dot(z, z)))

cos_d1_d3 = np.dot(x, z) / (np.sqrt(np.dot(x, x)) *
np.sqrt(np.dot(z, z)))


print("Cosine Similarity of D1 and D2: ", cos_d1_d2)

print("Cosine Similarity of D2 and D3: ", cos_d2_d3)

print("Cosine Similarity of D1 and D3: ", cos_d1_d3)
```

Output:

```
3 Documents
D1 = i love you
D2 = you are beautiful
D3 = i love beautiful koblenz
---
Cosine Similarity of D1 and D2:  0.33333333333333337
Cosine Similarity of D2 and D3:  0.2886751345948129
Cosine Similarity of D1 and D3:  0.5773502691896258
```